

Privacy Preserving Planning in Stochastic Environments

Guy Shani, Roni Stern, and Tommy Hefner

Ben Gurion University of the Negev, SISE Dept., Beer Sheva, Israel

Abstract

Collaborative privacy preserving planning (CPPP) has gained much attention in the past decade. To date, CPPP has focused on domains with deterministic action effects. In this paper, we extend CPPP to domains with stochastic action effects. We show how such environments can be modeled as an MDP. We then focus on the popular Real-Time Dynamic Programming (RTDP) algorithm for computing value functions for MDPs, extending it to the stochastic CPPP setting. We provide two versions of RTDP: a complete version identical to executing centralized RTDP, and an approximate version that sends significantly fewer messages and computes competitive policies in practice. We experiment on domains adapted from the deterministic CPPP literature.

1 Introduction

Designing autonomous agents that act collaboratively is an important goal. A fundamental requirement of such collaboration is to plan for multiple agents acting to achieve a common set of goals. *Collaborative privacy-preserving planning* (CPPP) is a multi-agent planning task in which agents need to achieve a common set of goals without revealing certain private information (?). In particular, in CPPP an individual agent may have a set of private facts and actions that it does not share with the other agents. CPPP has important motivating examples, such as planning for organizations that outsource some of their tasks.

In this paper we extend the CPPP framework to stochastic domains, where actions may have different effects with varying probabilities. In the planning community, stochastic domains are typically modeled using Markov decision processes (MDPs) (? , e.g.). We suggest an MDP formalization for privacy preserving stochastic problems.

A popular approach for solving goal based MDPs is the Real-Time Dynamic Programming (RTDP) algorithm (?), computing a value function, estimating the expected cost to the goal from each state. RTDP executes trajectories in the state space, updating the value function along the trajectory, and is guaranteed, under some restrictions, to converge to the optimal value function. We adapt RTDP to our stochastic CPPP setting, showing how agents execute jointly

state space trajectories, resulting in the distributed RTDP algorithm (DRTDP).

The complete version of DRTDP follows the same trajectories that will be computed on a centralized MDP. To do so, DRTDP requires a constant synchronization between the agents, which results in many messages. We thus suggest an approximate DRTDP version that requires synchronization only following public actions, which we call the Public Sync RTDP (PS-RTDP). This approximate DRTDP is not guaranteed to converge, and in particular, its trajectories may get into private cycles, requiring restarting the trajectory.

We compare the two versions over a set of domains adapted from the deterministic CPPP literature (?), showing that both versions converge to similar expected costs, while PS-RTDP requires as much as an order of magnitude fewer messages.

2 Background

We briefly review background on CPPP, MDPs, and RTDP.

Collaborative Privacy Preserving Planning An MA-STRIPS problem (?) is represented by a tuple $\langle P, \{A_i\}_{i=1}^k, I, G \rangle$ where: k is the number of agents, P is a finite set of primitive propositions (facts), A_i is the set of actions agent i can perform, I is the start state, and G is the goal condition.

Privacy-preserving MA-STRIPS extends MA-STRIPS by defining sets of variables and actions as private, known only to a single agent. $private_i(P)$ and $private_i(A_i)$ denote the variables and actions, respectively, that are private to agent i . $public(P)$ is the set of public facts in P . $public_i(A_i)$, the complement of $private_i(A_i)$ w.r.t. A_i , is the set of public actions of agent i . Some preconditions and effects of public actions may be private, and the action obtained by removing these private elements is called its *public projection*, and it is known to other agents. All agents are aware of the execution of a public action, and view the public effects of the action. Goals can be public, or private to one agent.

MAFS (?) is a popular algorithm for CPPP — a distributed algorithm in which every agent runs a best-first search to reach the goal. Agents maintain an open list of states, and in every iteration choose a state from the open list to expand, generating all its children and adding them to the open list (avoiding duplicates). Whenever an agent expands a state

that was generated by applying a public action, it also broadcasts this state to all other agents. An agent that receives a state adds it to the open list.

To preserve privacy, the private part of a state is obfuscated when broadcasting it, e.g., by replacing the private facts with some index, such that only the broadcasting agent knows how to map this state index to the corresponding private facts. Once the goal is reached, the agent achieving the goal informs all others, and the search process stops.

Markov Decision Processes In many domains there can be several alternative action effects. First, actions may fail, e.g., in the Logistics domain, loading a package may fail, and the package would remain where it was. Second, actions may have unwanted effects. For example, the driver may enter a wrong destination into the navigation app, and drive to the wrong location. In many cases one can have a stochastic model specifying the probability for each effect to occur.

Such planning domains can be modeled by Markov Decision Processes (MDPs) (?). In automated planning, an MDP is a tuple $\langle P, A, I, G, C \rangle$ where P is a finite set of propositions, A is a finite set of actions, I is the start state, G is the goal condition, and C is a cost function. The set of states of the MDP S is the set of all possible assignments to the propositions in P . An action $a \in A$ is defined by preconditions $pre(a)$ as above. However, the action effects $eff(a)$ is a set of stochastic effects $\langle \phi_i, p_i \rangle$, where ϕ_i is a conjunction of positive and negative literals, and p_i is the probability that this specific effect would occur. As some effect must always occur, $\sum_i p_i = 1$. For a pair of states s, s' , we denote by $tr(s, a, s')$ the probability that the agent would arrive at state s' by executing a at state s . If $s \models G$, then $tr(s, a, s) = 1, \forall a \in A$. That is, goal states are absorbing. The cost function assigns a cost to executing an action a at a state s . The cost for any action at a goal state $s \models G$ is 0, while executing an action at any other state incurs a positive cost. Hence, the agent strives to arrive at a goal state.

A solution for an MDP is a policy, a mapping $\pi : S \rightarrow A$, assigning an action to every state. A policy that minimizes the expected cost for reaching the goal is an optimal solution to an MDP. Many MDP algorithms compute a policy by first computing a value function $V : S \rightarrow \mathbb{R}$ assigning a value to each state, estimating the expected cost for achieving the goal. The value iteration algorithm computes a value function by iteratively applying the Bellman update:

$$Q(s, a) = C(s, a) + \sum_{s'} tr(s, a, s') V(s') \quad (1)$$

$$V(s) = \min_a Q(s, a) \quad (2)$$

over every state until convergence. The resulting policy is: $\pi^V(s) = \operatorname{argmin}_a C(s, a)$.

The RTDP algorithm (Algorithm 1) (? , e.g.) operates by launching simulated trajectories in state space, starting from the initial state. During a trajectory, at each state s , the algorithm selects a heuristically best action a , and then selects the next state from the $tr(s, a, \cdot)$ distribution. When RTDP reaches a goal state, the trajectory is terminated, and a new one is launched. Bellman updates are used to update the value function along the trajectory. If the value function

Algorithm 1: RTDP

```

1 RTDP()
2    $\forall s \in S, a \in A, Q(s, a) \leftarrow 0, V(s) \leftarrow 0$ 
3   while  $V$  has not converged do
4      $s \leftarrow s_0$ 
5     while  $s \not\models G$  do
6        $a^* \leftarrow \operatorname{argmin}_a Q(s, a)$ 
7        $Q(s, a) = C(s, a) + \sum_{s'} tr(s, a, s') V(s')$ 
8        $V(s) \leftarrow \min_{a'} Q(s, a')$ 
9       sample  $s'$  from  $tr(s, a^*, \cdot)$ 
10     $s \leftarrow s'$ 
```

is initialized optimistically (e.g. to 0), RTDP converges to the optimal value function.

3 Privacy Preserving MDPs

We extend the definition of CPPP to stochastic domains, modeled as an MDP, with each agent viewing only a part of the complete MDP. More formally, a stochastic collaborative privacy preserving planning problem (SCPPP) is a tuple: $\langle P, \{A_i\}_{i=1}^k, tr, I, G, C \rangle$ where k is the number of agents, P is a finite set of primitive propositions (facts), A_i is the set of actions agent i can perform, I is the start state, G is the goal condition, and C is a cost function. As in a CPPP problem, $private_i(P)$ and $private_i(A_i)$ denote private variables and action of agent i . $public(P)$ is the set of public facts in P , and $public_i(A_i)$ is the set of public actions of agent i . As in an MDP, actions have stochastic effects, i.e. $eff(a) = \{ \langle \phi_i, p_i \rangle \}$ where p_i is the probability that effect ϕ_i would occur. As in an MDP, C assigns a cost for executing an action at a state. tr is a transition function, specifying the probability of moving from states s to s' using action a .

While in classical CPPP problems a solution is a sequence of public and private actions, in an SCSPP problem the solution is a policy, determining at each step which agent should execute an action, and which action must be executed. Hence, the policy of an agent i is a mapping $\pi_i : S_i \rightarrow A_i \cup \{noop\}$, where S_i is the set of local states — assignments to the private propositions of i and the public propositions. Such a policy either chooses an action to execute at a given state, or lets another agent perform an action.

In this paper we focus on policies that do not allow parallel execution, that is, for each state s of the system, there is exactly one agent where $\pi_i(s_i) \neq noop$, where s_i is the local view of agent i of the global state s . We leave extensions that allow for parallel executions where more than one agent executes an action at each step to future research.

Complete DRTDP We now present our distributed RTDP (DRTDP) algorithm for solving privacy preserving MDP problems. DRTDP operates by running a trajectory that is advanced by one agent at a time. That is, similar to the forward search of MAFS, the trajectory begins with one agent but can then be advanced by other agents. As in MAFS, DRTDP requires message passing between the agents. Like MAFS, messages contain a public state, and indexes of private states. However, as opposed to MAFS, messages also

Algorithm 2: DRTDP for agent i

```

1 DRTDP( $i$ )
2    $\forall s \in S, a \in A, Q_i(s, a) \leftarrow 0, V_i(s) \leftarrow 0$ 
3   while true do
4     process-messages()
5     if  $s_c^i \neq \text{null}$  then
6       advance-trajectory()
7   process-message()
8   foreach  $\text{Message } m = \langle s, t, v, j \rangle$  do
9     if  $m.\text{type} = Q\text{-value request}$  then
10      send to  $m.j$   $\langle m.s, Q\text{-value response}, V_i(s), i \rangle$ 
11     if  $m.\text{type} = Q\text{-value response}$  then
12       if  $m.v < V_i(s)$  then
13          $V_i(s) = m.v, \text{best}_i(s) \leftarrow m.j$ 
14       if  $m.\text{type} = \text{trajectory}$  then
15          $s_c^i \leftarrow m.s$ 
16   advance-trajectory()
17    $a^* = \text{argmin}_a Q_i(s_c^i, a)$ 
18   update( $s_c^i, a^*$ )
19   sample  $s'$  from  $tr(s_c^i, a^*, \cdot)$ 
20    $s_c^i \leftarrow \text{choose-next-agent}(s')$ 
21   choose-next-agent( $s$ )
22   if  $s \subseteq G$  then
23     broadcast & wait  $\langle s_0, Q\text{-value request}, \times, i \rangle$ 
24      $s \leftarrow s_0$ 
25   if  $\text{best}(s) = i$  then return  $s$ 
26   else send to  $\text{best}(s)$   $\langle s, \text{trajectory}, \times, i \rangle$ , return null
27   update( $s, a$ )
28   foreach  $s' \in tr(s, a, \cdot)$  do
29     broadcast & wait  $\langle s', Q\text{-value request}, \times, i \rangle$ 
30      $Q_i(s, a) = C(s, a) + \sum_{s'} tr(s, a, s') V_i(s')$ 
31      $V_i(s) \leftarrow \min_{a'} Q_i(s, a')$ 

```

contain the current values for the states in the message.

In the complete DRTDP, at each step, a single agent is responsible for advancing the trajectory. This agent requests all other agents to send it their estimated cost to the goal from the current state for their best action. The agent sending the best expected cost is chosen to execute its action and select the next state. This is repeated until the goal is reached. Each agent i maintains the functions $Q_i(s, a)$ and $V_i(s)$.

Algorithm 2 presents a simplified version of our distributed RTDP method. Each agent continuously processes all received messages (line 4). A message m is a tuple $\langle s, t, v, i \rangle$ where s is a state, represented by the public facts and the private state indexes of all agents, v is a (possibly empty) value, i the sending agent, and t is the message type. There can be 3 types of messages: (1) Q -value request: agent i is requesting all agents to send their values for the state s . (2) Q -value response: agent i is answering a Q -value request message with its value for the state s . The receiving agent checks if it has received a better value, and if so, changes $V_i(s)$, also recording the sending agent in $\text{best}_i(s)$ (line 13). (3) Trajectory: agent i transfers responsibility of advancing the trajectory from s to the receiving agent.

The private variable s_c^i maintains the current state of the trajectory for agent i , which can be null , if the trajectory is currently under the responsibility of another agent. If

Algorithm 3: PS-RTDP for agent i

```

1 advance-trajectory()
2    $a^* = \min_a Q_i(s_c^i, a)$ 
3   sample  $s'$  from  $tr(s_c^i, a^*, \cdot)$ 
4   if private cycle detected then
5     restart trajectory
6   if  $a^*$  is a private action then
7     local-update( $s_c^i, a^*$ ),  $s_c^i \leftarrow s'$ 
8   else
9     update( $s_c^i, a^*$ ),  $s_c^i \leftarrow \text{choose-next-agent}(s')$ 
10  local-update( $s, a$ )
11   $Q_i(s, a) = C(s, a) + \sum_{s'} tr(s, a, s') \min_{a'} Q_i(s', a')$ 
12   $V_i(s) \leftarrow \min_a Q_i(s, a)$ 

```

$s_c^i \neq \text{null}$, then i is responsible for advancing the trajectory (line 6). When receiving a trajectory message, s_c^i is set to the received state. When agent i receives responsibility for a trajectory, it first selects its best action a^* (line 17). At this point, we can limit our attention to the actions of agent i , because we already compared the values from all other agents, and i reported the best expected cost.

We then update the Q -function and the value function. The only Q -value that changes is $Q_i(s_c^i, a^*)$ of agent i whose action is executed. Hence, agent i requests (line 29) all other agents to send their values for all possible next states after executing a^* at s_c^i . After all values have been received, we update $Q_i(s_c^i, a^*)$ and $V_i(s_c^i)$ (line 30).

After updating the value function, the agent selects the next possible state s' , and then must choose the agent that receives the responsibility to advance the trajectory for s' . First, if s' is a goal state, then the agent must start a new trajectory. To do that, the agent sends a request to all agents for their best value for the initial state s_0 . If s' is not a goal state, recall that during the value function update the agent has already requested values from all other agents for all possible next states, including s' (line 29). Hence, agent i already maintains the best agent to handle s' in $\text{best}_i(s')$. If the best next agent is i , then it sets s_c^i to s' . Otherwise, it sends a trajectory message to $\text{best}_i(s')$, transferring responsibility for the trajectory, and sets s_c^i to null . This algorithm results in identical trajectories to the ones generated by RTDP on the joint problem. Thus, the cost added for preserving privacy is only the cost of the message passing mechanism.

Public Synchronization RTDP The approximate version of DRTDP, which we call Public Sync RTDP (PS-RTDP), relies on the intuition that often there is no need to interleave the private actions of different agents. That is, as in MAFS, following a private action of agent i , the next action should also be of agent i . Hence, in PS-RTDP, an agent chosen to execute the next action continues to execute additional actions until it executes a public action. Then, the agents vote, as in the complete DRTDP on which agent takes ownership of the trajectory and progresses it forward. Algorithm 3 shows the difference between the algorithms. In PS-RTDP, the agent responsible for advancing a trajectory considers only its own actions. If the selected action is private, then the agent updates the value using only its own Q -values (line 7), and remains responsible for the tra-

Domain	# Actions	# Facts	Best Cost	Expansions $\times 10^4$	Messages $\times 10^4$	# trajectories + restarts	Total Time (sec)
blocks-2-2	26	26	(4.58 / 4.52)	(0.025 / 0.045)	(0.135 / 0.038)	(40 / 40 + 41)	(0.068 / 0.062)
blocks-3-3	129	63	(7.48 / 7.44)	(0.120 / 0.202)	(1.274 / 0.455)	(60 / 90 + 127)	(0.516 / 0.473)
blocks-4-3	315	99	(9.7 / 9.72)	(0.713 / 0.840)	(7.636 / 2.394)	(80 / 130 + 224)	(4.1 / 3.5)
blocks-5-2	422	107	(11.98 / 12.36)	(5.247 / 5.412)	(30.434 / 9.366)	(150 / 380 + 691)	(35.3 / 29.8)
blocks-6-2	746	146	(14.8 / 16.5)	(68.322 / 68.977)	(399.718 / 117.096)	(900 / 2140 + 7057)	(497.8 / 427.7)
depot-2-3	69	49	(11.2 / 11.4)	(0.276 / 0.973)	(2.844 / 3.219)	(90 / 650 + 62)	(1.2 / 3.0)
depot-3-3	149	74	(13.48 / 13.22)	(2.894 / 3.152)	(29.708 / 8.126)	(220 / 160 + 483)	(22.7 / 15.6)
depot-3-4	334	91	(16.76 / 16.76)	(9.273 / 11.765)	(141.352 / 44.033)	(410 / 240 + 2515)	(79.8 / 55.8)
depot-2-5	245	71	(11.4 / 11.14)	(4.342 / 5.050)	(89.343 / 23.417)	(350 / 170 + 1389)	(55.7 / 23.9)
depot-3-5	407	113	(16.78 / 16.78)	(55.045 / 59.236)	(1136.297 / 289.422)	(2330 / 740 + 10437)	(715.4 / 344.8)
logistics-1-3	49	27	(12.78 / 12.62)	(0.371 / 0.191)	(4.208 / 0.164)	(110 / 40 + 102)	(1.1 / 0.217)
logistics-2-3	69	34	(20.38 / 20.78)	(4.203 / 2.516)	(47.815 / 3.494)	(440 / 140 + 792)	(13.9 / 3.5)
logistics-2-4	80	40	(20.82 / 20.84)	(13.437 / 7.174)	(224.140 / 14.711)	(1130 / 170 + 2440)	(54.4 / 10.9)
logistics-2-5	203	70	(27.0 / 21.58)	(122.934 / 40.523)	(2705.937 / 103.567)	(2134 / 220 + 14162)	(848.5 / 81.7)
logistics-3-4	104	48	(26.8 / 27.22)	(152.786 / 109.649)	(2570.205 / 250.775)	(8010 / 770 + 27091)	(720.6 / 186.0)

Table 1: Empirical comparison of the DRTDP and PS-RTDP. Cell format is DRTDP / PS-RTDP.

jectory. If a^* is a public action, however, the agent uses the value function update mechanism, and selects the next agent to take responsibility as in Algorithm 2 (line 27). As all messages are sent by the global update mechanism and the next agent selection process, reducing calls to these phases reduces the number of messages significantly.

PS-RTDP is no longer guaranteed to converge. One reason is that an agent chosen to advance the trajectory may get stuck in a loop of private actions, without being able to reach the goal, or execute any public action. For example, in the Logistics domain, agent 1 may unload a package at location A . As all agents currently have Q -values of 0, the tie breaking mechanism decided that an agent with no access to A takes the trajectory. That agent has no public actions available, and can hence only advance the trajectory by private driving actions, never releasing responsibility to other agents. We therefore add a private cycle detection mechanism (line 4). If a cycle is detected, we restart the trajectory from s_0 . The sensitivity of the cycle detection is important, as cycles may arise due to stochastic effects. Detecting a cycle too soon may make the algorithm ignorant to a possible path towards the goal. Detecting a cycle too late may cause the algorithm to be much slower than the complete DRTDP.

4 Empirical Analysis

We now provide empirical analysis on domains adapted from the CPPP literature, comparing the complete DRTDP and the approximate DRTDP algorithms, implemented in Python. All experiments were run on Google cloud running a Xeon CPU with two 2GHz cores, and 1.8GB user RAM. The domains were adapted to the algorithms after 10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 110, 120, 130, 140, 150, 160, 170, 180, 190, 200, 210, 220, 230, 240, 250, 260, 270, 280, 290, 300, 310, 320, 330, 340, 350, 360, 370, 380, 390, 400, 410, 420, 430, 440, 450, 460, 470, 480, 490, 500, 510, 520, 530, 540, 550, 560, 570, 580, 590, 600, 610, 620, 630, 640, 650, 660, 670, 680, 690, 700, 710, 720, 730, 740, 750, 760, 770, 780, 790, 800, 810, 820, 830, 840, 850, 860, 870, 880, 890, 900, 910, 920, 930, 940, 950, 960, 970, 980, 990, 1000, 1010, 1020, 1030, 1040, 1050, 1060, 1070, 1080, 1090, 1100, 1110, 1120, 1130, 1140, 1150, 1160, 1170, 1180, 1190, 1200, 1210, 1220, 1230, 1240, 1250, 1260, 1270, 1280, 1290, 1300, 1310, 1320, 1330, 1340, 1350, 1360, 1370, 1380, 1390, 1400, 1410, 1420, 1430, 1440, 1450, 1460, 1470, 1480, 1490, 1500, 1510, 1520, 1530, 1540, 1550, 1560, 1570, 1580, 1590, 1600, 1610, 1620, 1630, 1640, 1650, 1660, 1670, 1680, 1690, 1700, 1710, 1720, 1730, 1740, 1750, 1760, 1770, 1780, 1790, 1800, 1810, 1820, 1830, 1840, 1850, 1860, 1870, 1880, 1890, 1900, 1910, 1920, 1930, 1940, 1950, 1960, 1970, 1980, 1990, 2000, 2010, 2020, 2030, 2040, 2050, 2060, 2070, 2080, 2090, 2100, 2110, 2120, 2130, 2140, 2150, 2160, 2170, 2180, 2190, 2200, 2210, 2220, 2230, 2240, 2250, 2260, 2270, 2280, 2290, 2300, 2310, 2320, 2330, 2340, 2350, 2360, 2370, 2380, 2390, 2400, 2410, 2420, 2430, 2440, 2450, 2460, 2470, 2480, 2490, 2500, 2510, 2520, 2530, 2540, 2550, 2560, 2570, 2580, 2590, 2600, 2610, 2620, 2630, 2640, 2650, 2660, 2670, 2680, 2690, 2700, 2710, 2720, 2730, 2740, 2750, 2760, 2770, 2780, 2790, 2800, 2810, 2820, 2830, 2840, 2850, 2860, 2870, 2880, 2890, 2900, 2910, 2920, 2930, 2940, 2950, 2960, 2970, 2980, 2990, 3000, 3010, 3020, 3030, 3040, 3050, 3060, 3070, 3080, 3090, 3100, 3110, 3120, 3130, 3140, 3150, 3160, 3170, 3180, 3190, 3200, 3210, 3220, 3230, 3240, 3250, 3260, 3270, 3280, 3290, 3300, 3310, 3320, 3330, 3340, 3350, 3360, 3370, 3380, 3390, 3400, 3410, 3420, 3430, 3440, 3450, 3460, 3470, 3480, 3490, 3500, 3510, 3520, 3530, 3540, 3550, 3560, 3570, 3580, 3590, 3600, 3610, 3620, 3630, 3640, 3650, 3660, 3670, 3680, 3690, 3700, 3710, 3720, 3730, 3740, 3750, 3760, 3770, 3780, 3790, 3800, 3810, 3820, 3830, 3840, 3850, 3860, 3870, 3880, 3890, 3900, 3910, 3920, 3930, 3940, 3950, 3960, 3970, 3980, 3990, 4000, 4010, 4020, 4030, 4040, 4050, 4060, 4070, 4080, 4090, 4100, 4110, 4120, 4130, 4140, 4150, 4160, 4170, 4180, 4190, 4200, 4210, 4220, 4230, 4240, 4250, 4260, 4270, 4280, 4290, 4300, 4310, 4320, 4330, 4340, 4350, 4360, 4370, 4380, 4390, 4400, 4410, 4420, 4430, 4440, 4450, 4460, 4470, 4480, 4490, 4500, 4510, 4520, 4530, 4540, 4550, 4560, 4570, 4580, 4590, 4600, 4610, 4620, 4630, 4640, 4650, 4660, 4670, 4680, 4690, 4700, 4710, 4720, 4730, 4740, 4750, 4760, 4770, 4780, 4790, 4800, 4810, 4820, 4830, 4840, 4850, 4860, 4870, 4880, 4890, 4900, 4910, 4920, 4930, 4940, 4950, 4960, 4970, 4980, 4990, 5000, 5010, 5020, 5030, 5040, 5050, 5060, 5070, 5080, 5090, 5100, 5110, 5120, 5130, 5140, 5150, 5160, 5170, 5180, 5190, 5200, 5210, 5220, 5230, 5240, 5250, 5260, 5270, 5280, 5290, 5300, 5310, 5320, 5330, 5340, 5350, 5360, 5370, 5380, 5390, 5400, 5410, 5420, 5430, 5440, 5450, 5460, 5470, 5480, 5490, 5500, 5510, 5520, 5530, 5540, 5550, 5560, 5570, 5580, 5590, 5600, 5610, 5620, 5630, 5640, 5650, 5660, 5670, 5680, 5690, 5700, 5710, 5720, 5730, 5740, 5750, 5760, 5770, 5780, 5790, 5800, 5810, 5820, 5830, 5840, 5850, 5860, 5870, 5880, 5890, 5900, 5910, 5920, 5930, 5940, 5950, 5960, 5970, 5980, 5990, 6000, 6010, 6020, 6030, 6040, 6050, 6060, 6070, 6080, 6090, 6100, 6110, 6120, 6130, 6140, 6150, 6160, 6170, 6180, 6190, 6200, 6210, 6220, 6230, 6240, 6250, 6260, 6270, 6280, 6290, 6300, 6310, 6320, 6330, 6340, 6350, 6360, 6370, 6380, 6390, 6400, 6410, 6420, 6430, 6440, 6450, 6460, 6470, 6480, 6490, 6500, 6510, 6520, 6530, 6540, 6550, 6560, 6570, 6580, 6590, 6600, 6610, 6620, 6630, 6640, 6650, 6660, 6670, 6680, 6690, 6700, 6710, 6720, 6730, 6740, 6750, 6760, 6770, 6780, 6790, 6800, 6810, 6820, 6830, 6840, 6850, 6860, 6870, 6880, 6890, 6900, 6910, 6920, 6930, 6940, 6950, 6960, 6970, 6980, 6990, 7000, 7010, 7020, 7030, 7040, 7050, 7060, 7070, 7080, 7090, 7100, 7110, 7120, 7130, 7140, 7150, 7160, 7170, 7180, 7190, 7200, 7210, 7220, 7230, 7240, 7250, 7260, 7270, 7280, 7290, 7300, 7310, 7320, 7330, 7340, 7350, 7360, 7370, 7380, 7390, 7400, 7410, 7420, 7430, 7440, 7450, 7460, 7470, 7480, 7490, 7500, 7510, 7520, 7530, 7540, 7550, 7560, 7570, 7580, 7590, 7600, 7610, 7620, 7630, 7640, 7650, 7660, 7670, 7680, 7690, 7700, 7710, 7720, 7730, 7740, 7750, 7760, 7770, 7780, 7790, 7800, 7810, 7820, 7830, 7840, 7850, 7860, 7870, 7880, 7890, 7900, 7910, 7920, 7930, 7940, 7950, 7960, 7970, 7980, 7990, 8000, 8010, 8020, 8030, 8040, 8050, 8060, 8070, 8080, 8090, 8100, 8110, 8120, 8130, 8140, 8150, 8160, 8170, 8180, 8190, 8200, 8210, 8220, 8230, 8240, 8250, 8260, 8270, 8280, 8290, 8300, 8310, 8320, 8330, 8340, 8350, 8360, 8370, 8380, 8390, 8400, 8410, 8420, 8430, 8440, 8450, 8460, 8470, 8480, 8490, 8500, 8510, 8520, 8530, 8540, 8550, 8560, 8570, 8580, 8590, 8600, 8610, 8620, 8630, 8640, 8650, 8660, 8670, 8680, 8690, 8700, 8710, 8720, 8730, 8740, 8750, 8760, 8770, 8780, 8790, 8800, 8810, 8820, 8830, 8840, 8850, 8860, 8870, 8880, 8890, 8900, 8910, 8920, 8930, 8940, 8950, 8960, 8970, 8980, 8990, 9000, 9010, 9020, 9030, 9040, 9050, 9060, 9070, 9080, 9090, 9100, 9110, 9120, 9130, 9140, 9150, 9160, 9170, 9180, 9190, 9200, 9210, 9220, 9230, 9240, 9250, 9260, 9270, 9280, 9290, 9300, 9310, 9320, 9330, 9340, 9350, 9360, 9370, 9380, 9390, 9400, 9410, 9420, 9430, 9440, 9450, 9460, 9470, 9480, 9490, 9500, 9510, 9520, 9530, 9540, 9550, 9560, 9570, 9580, 9590, 9600, 9610, 9620, 9630, 9640, 9650, 9660, 9670, 9680, 9690, 9700, 9710, 9720, 9730, 9740, 9750, 9760, 9770, 9780, 9790, 9800, 9810, 9820, 9830, 9840, 9850, 9860, 9870, 9880, 9890, 9900, 9910, 9920, 9930, 9940, 9950, 9960, 9970, 9980, 9990, 10000. We used a random seed for each domain and problem. The results are shown in Table 1. The approximate DRTDP algorithm is faster than the complete DRTDP algorithm in all domains. For example, in the blocks domain, the approximate DRTDP requires fewer goal reaching trajectories, not counting restarts after loops, that can be much shorter.

In the blocks domain, the approximate RTDP generated about one third of the messages. On the other hand, it often required many more trajectories to converge, especially for the larger problems, and hence resulted in only slightly lower runtime. Only in the largest blocks problem, PS-RTDP resulted in significantly higher average cost. This is because in blocks world there is a relatively small number of private actions, and hence the public version is not very different.

In Logistics, the PS-RTDP sends in many cases fewer than $\frac{1}{10}$ of the messages, and is hence considerably faster on many problems. In Logistics, PS-RTDP also required much fewer trajectories to converge. In Depot, as in blocks world, the number of messages sent by PS-RTDP is about one third of the messages sent by the complete DRTDP. In this domain, however, the approximate version required about half the time before converging, especially on larger problems.

5 Conclusion and Future Work

We presented two algorithms for CPPP in stochastic environments. DRTDP is a distributed, privacy preserving adaptation of RTDP, which is identical in execution to RTDP on the joint problem. PS-RTDP is an approximation of DRTDP that shares only public changes between agents to reduce the amount of messages during planning. Experiments show that PS-RTDP sends significantly less messages than DRTDP, sending an order of magnitude fewer messages in some cases. Future research can advance multiple trajectories simultaneously, allowing agents that do not currently advance a trajectory to start a new one. sequential action execution. We will investigate an extension to concurrent execution.

ea fuga nihil illo excepturi similique enim alias obcaecati voluptatum, laborum itaque qui saepe rerum?